

Chapter 10

Contents

Chapters

- 10. PIC 18 Parallel Slave Port, Serial port & its programming
 - 10.1 Parallel Slave Port
 - 10.2 Introduction to PIC18 Serial Communication
 - 10.3 USART Asynchronous Mode
 - 10.3.1 Transmit
 - 10.3.2 Receive
 - 10.4 USART Synchronous Mode
 - 10.4.1 Transmit
 - 10.4.2 Receive

10.1 Parallel Slave Port

The PIC18F458 & PIC18F8720 consists of one Parallel Slave Port - PSP, while no such feature exists in PIC18C801. In PIC18F8720, the PSP feature is available only in microcontroller mode. The parallel slave port of PIC18 as the name suggests is a slave port that is used for parallel transfer of data. It acts as a slave and can be directly interfaced to a data bus of microprocessor or another microcontroller which may act as a master.

The PIC18's devices such as PIC18C801 and PIC18F8720 can act as a master as they contain external memory interface including external read, write signals and address bus using which, the chip select could be generated and a slave PSP could be interfaced.

Traditional microprocessors like 8085, 8086 use 8255 for such data transfers similar to any microprocessors. Additionally, most microcontrollers can be used as master through the Port I/O pins and Interrupts. The feature of parallel slave port on PIC18 in addition to two I/O pins for selection and interrupts of PORTB for handshake provides the flexibility to use PIC18 as an advanced 8255 – Programmable Peripheral Interface. However, it requires programming of PIC to handle the handshake signals.

The parallel slave port consists of 8 Data lines, 3 control signal lines, in addition to interrupt flag, two flags (input buffer and output buffer) that are similar to 8255, an overflow flag similar to over-run error bit of serial communication and PSP enable bit. The summary of pins associated PSP are presented in Table 10.1 and the block diagram of PSP is shown in fig 10.1. A detailed structure for PORTD of both devices is presented in fig 2.

D0:D7 – Parallel Data lines, RD0:RD7:

The PSP consists of 8 lines that act as data lines for parallel data transfer. They are associated with PORT D, RD0:RD7.

Read – RD:

The master – either the microprocessor or a microcontroller generates this signal to read a data byte from PIC18's Parallel Slave Port. The RD signal is associated with 0th pin of PORTE the RE0 pin. A read signal is effective only when the chip select is active. i.e. the PSP is read only when CS input signal and RD signal is 0.

Write – WR:

The master – either the microprocessor or a microcontroller generates this signal to write a data byte into PIC18's Parallel Slave Port. The WR signal is associated with 1st pin of PORTE the RE1 pin. A write signal is effective only when the chip select is active. i.e. the PSP is written only when CS input signal and WR signal is 0.

Chip Select - CS:

The master – either the microprocessor or a microcontroller typically generates the PSP's assigned address on the external memory address bus. This address is used to generate the required chip select signal of the PSP. The CS signal is associated with 2nd pin of PORTE the RE2 pin. A sample chip-select signal generation for memory, I/Os is shown in example 3, fig 7.35 of memory interfacing section in chapter 7.7.5.

Table 10.1: Summary of Pins associated with PSP of PIC18F458 & PIC18F8720

Function	PSP Name	Port Pins	Other Functions	
			PIC18F458	PIC18F8720
Parallel Data Lines	D0:D7	RD0:RD7	Comparator inputs, ECCP1 PWM Outputs	External Memory Access - AD0:AD7
Read	<u>RD</u>	RE0	ADC: AN5	AD8
Write	<u>WR</u>	RE1	ADC: AN6, C1OUT	AD9
Chip Select	<u>CS</u>	RE2	ADC: AN7, C2OUT	AD10

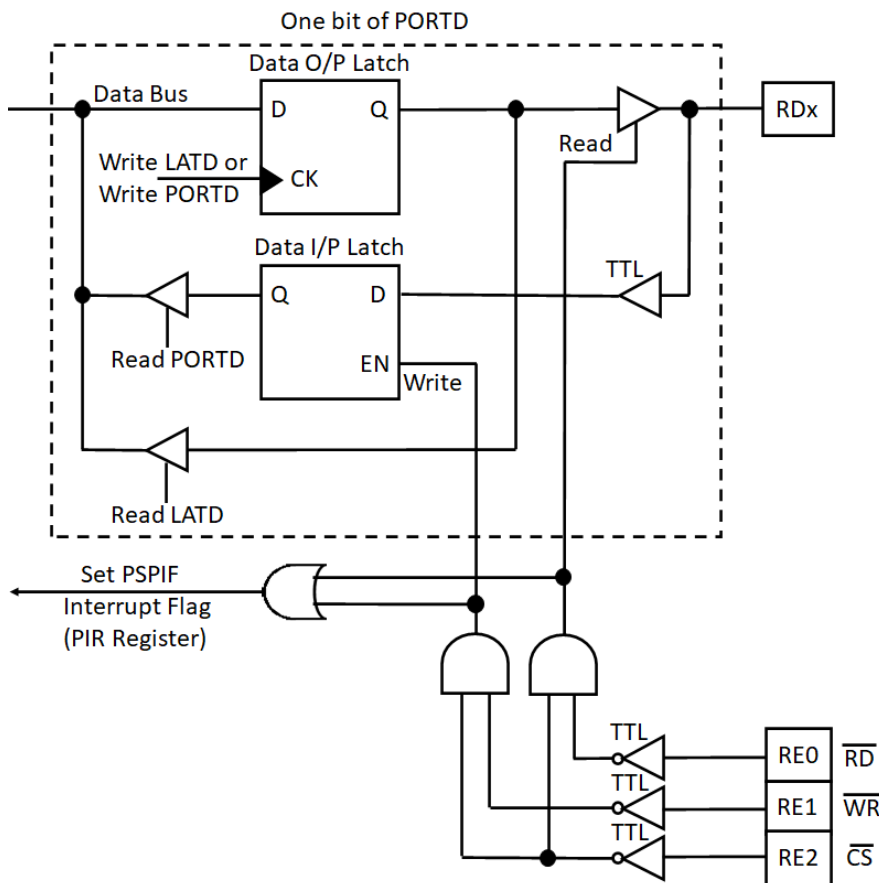


Fig 10.1 Simplified Block Diagram of PSP

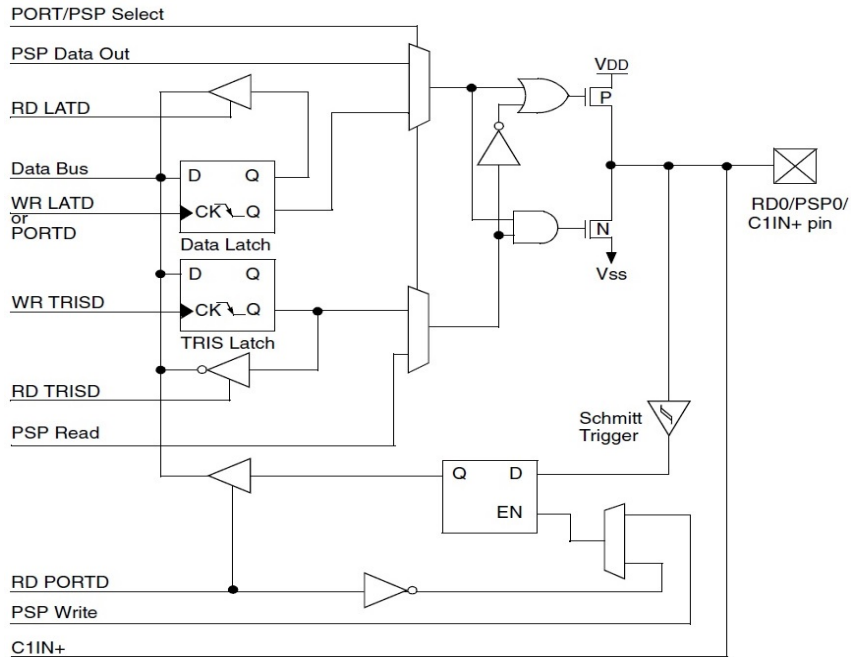


Fig 10.2a PORTD I/O structure of PIC18F458

(Source: PIC18FXX8 Data Sheet - DS41159E)

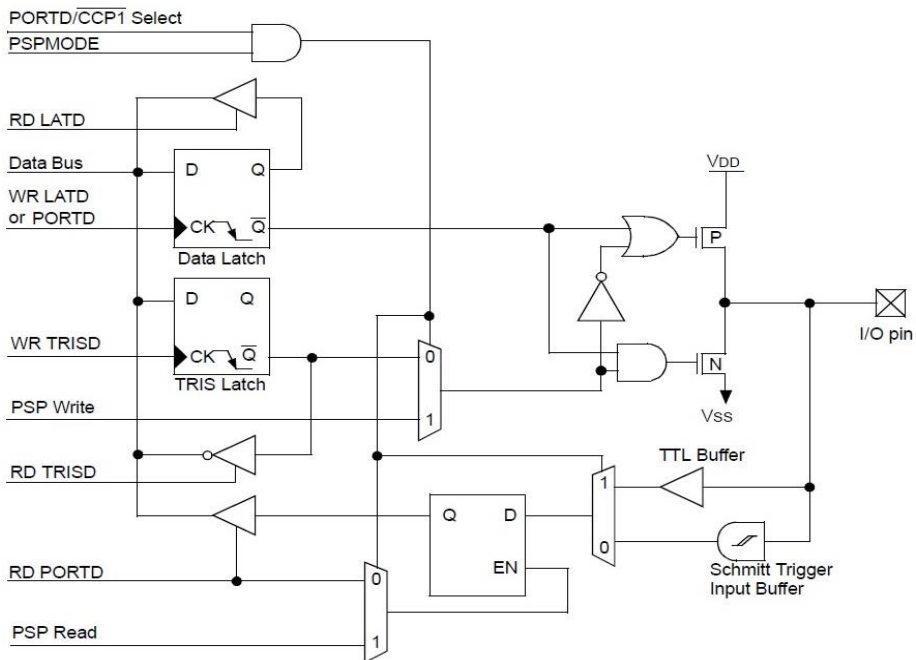


Fig 10.2b PORTD I/O structure of PIC18F8720

(Source: PIC18F8720 Data Sheet - DS39609C)

The PORTE is multiplexed with other functions as shown in table 10.1. The PORTE pins are used as control inputs as the PSP is a slave and is controlled by external microprocessor or microcontroller. Hence the RE0:RE2 pins must be configured as input pins by setting the TRISE<0:2> bits to 0. Additionally, in PIC18F458 the PCFG2:PCFG0 bits of ADCON1<3:0> register must be set such that RE0:RE2 are configured as digital I/Os.

The PORTD is commonly used by both the PIC18F458 and PIC18F8720. The PORTE varies between the devices. The registers associated with PSP for both devices are presented in Table 10.2.

The PIC18F458 has only 3 PORTE pins - RE0:RE2, however its direction control register TRISE is an 8-bit register that contains the PSP status and PSP enable bits along with direction controls for RE0:RE2.

The PIC18F8720 has 8 pins of PORTE – RE0:RE7 and their direction controls in 8-bit TRISE register. However, a separate PSPCON register is provided that includes the PSP status bits and PSP enable/select bit. The details of the PSP Enable/select, and Status bits are presented below.

PSP Mode Select bit – PSPMODE:

The parallel slave port is enabled by setting the PSPMODE select bit of TRIS<4> to a 1 in PIC18F458 or PSPMODE select bit of PSPCON<4> to a 1 in PIC18F8720. When this bit is set to 1, the 8-bit wide PORTD is configured as parallel slave data port for data transfer and PORTE bits RE0:RE2 are configured as PSP control signals – Read: RD, Write: WR, Chip Select: CS. The Port D is asynchronously readable or writable by the PSP control signals.

Output Buffer Full Status bit – OBF:

The OBF – Output Buffer Full Status bit indicates the status of output buffer. A negative edge (active low) read signal from the external master causes the OBF bit to be cleared to 0. This causes the PORTD data to be placed on the bus which is read by the external master. The OBF bit is set to 1 when the output buffer is loaded with a new value, i.e. it indicates that the output buffer still holds the previously written word. The PSP read waveform is shown in figure 10.3a. The OBF status bit is associated with 6th pin of TRISE register in PIC18F458 or 6th pin of PSPCON register in PIC18F8720 and is set or cleared automatically.

Input Buffer Full Status bit – IBF:

The IBF – Input Buffer Full Status bit indicates the status of input buffer. The master places the data on the data bus/PORTD and asserts a negative edge (active low) write signal. The data written by external master is loaded into the input buffer and must be read by PIC18's CPU. A positive edge (high) write signal from master sets the IBF bit to 1 to indicate

that the PIC18 CPU must read the data. The IBF bit is set to 0 when the data is read by PIC18 indicating no new data is received. The PSP write waveform is shown in figure 10.3b. The IBF status bit is associated with 7th pin of TRISE register in PIC18F458 or 7th pin of PSPCON register in PIC18F8720 and is set or cleared automatically.

Table 10.2: PSP related registers of PIC18F458 & PIC18F8720

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Common to PIC18F458 & PIC18F8720								
TRISD	PORTD Direction Control Register							
LATD	LATD Output Latch Register							
PORTD	RD7	RD6	RD5	RD4	RD3	RD2	RD1	RD0
Specific to PIC18F458								
TRISE	IBF	OBF	IBOV	PSP MODE	—	PORTE Direction Control Bits		
LATE	—	—	—	—	—	LATE Output Latch Bits		
PORTE	—	—	—	—	—	RE2	RE1	RE0
Specific to PIC18F8720								
TRISE	PORTE Direction Control Register							
LATE	LATE Output Latch Register							
PORTE	RE7	RE6	RE5	RE4	RE3	RE2	RE1	RE0
PSPCON	IBF	OBF	IBOV	PSPMODE	—	—	—	—

Input Buffer Overflow Detect bit – IBOV:

The IBOV – Input Buffer Overflow Detect bit indicates the occurrence of input buffer overflow. When the IBF is set, the PIC18 must read the data from the input buffer. Before the PIC18 reads the data available in the input buffer, if a new data is written by the external master, the previous unread data is overwritten with this new data. This causes the IBOV status bit to be set indicating that previous data is overwritten with new data. This bit is typically 0 when no overflow occurs and must be cleared to 0 in software when overflow occurs. The IBOV bit is similar to OERR error bit in serial communication. The IBOV status bit is associated with 5th pin of TRISE register in PIC18F458 or 5th pin of PSPCON register in PIC18F8720.

Table 10.3: PSP Interrupt bits & registers of PIC18F458 & PIC18F8720

Interrupt Name	Flag	Flag Reg	Enable	Enable Reg	Priority	Priority Reg
PIC18F458 & PIC18F8720	PSPIF	PIR1	PSPIE	PIE1	PSPIP	IPR1
PIC18C801	-	-	-	-	-	-

The details of the PSP interrupt related bits and their corresponding registers are presented in Table 10.3. It must be noted that the PSP read or write operation does not have separate interrupt flags. Both the operations trigger the setting of the same PSPIF flag.

Hence the user software must determine if the interrupt is due to the read or write operation by checking the IBF and OBF status bits.

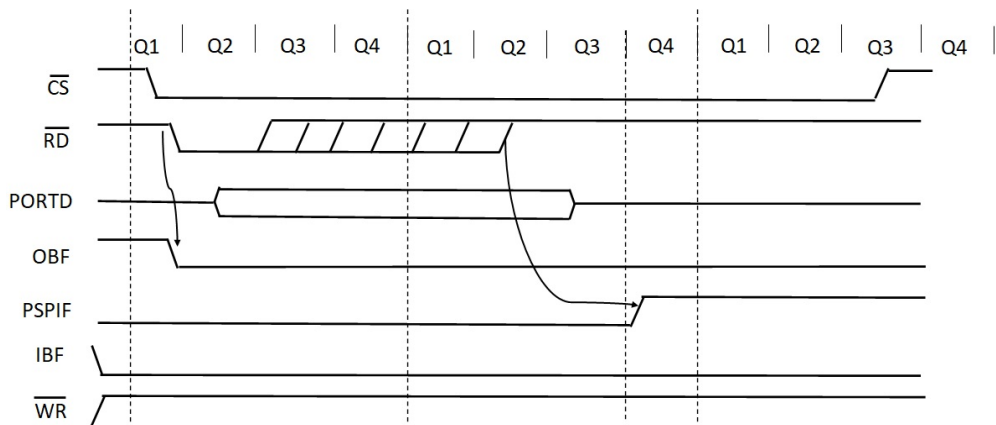


Fig 10.3a PSP Read Waveform

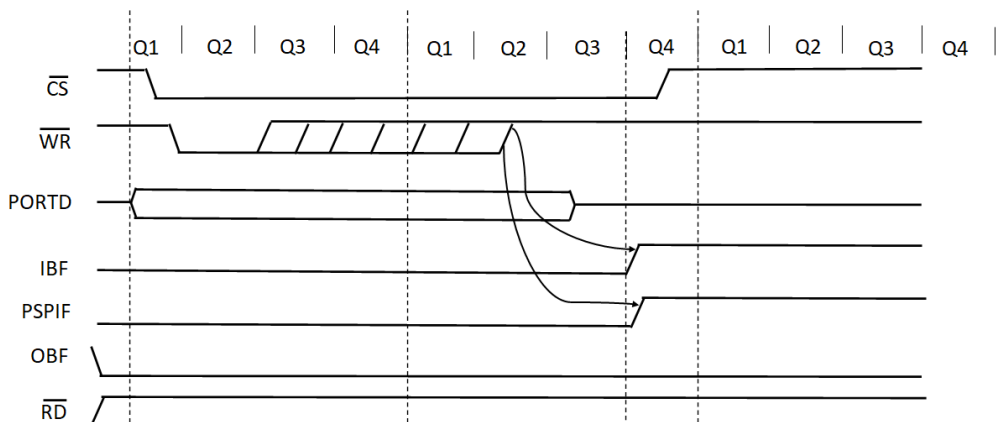


Fig 10.3b PSP Write Waveform

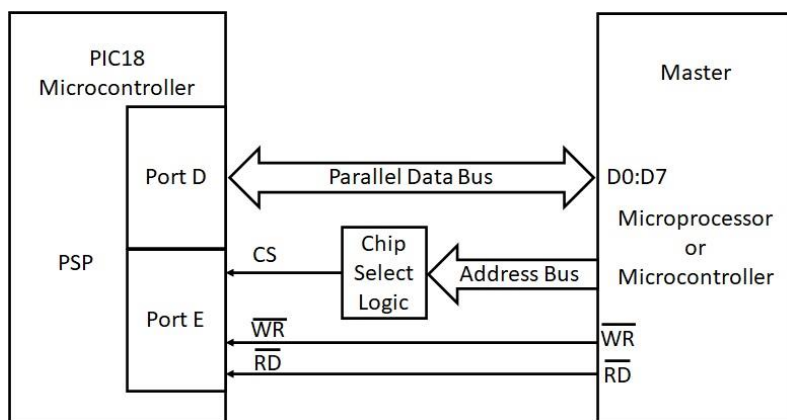


Fig 10.4 Interfacing PIC18 PSP with a Master Microprocessor

The disabling or positive edge of a read or write signal by the external master causes the PSPIF flag to be set and may generate an interrupt if the PSPIE bit of PIR1 register and global interrupts are enabled along with appropriate priority setting. A simple PSP interfacing circuit is shown in fig 10.4.

10.2 Introduction to PIC18 Serial Communication

The PIC18F458 consists of three serial communication modules (USART, MSSP & CAN), whereas the PIC18C801 contains only two (USART & MSSP). The PIC18F8720 contains two types of serial communication modules (2 USART, 1 MSSP). The summary of serial communication modules is presented in Table 10.4.

- USART – Universal Synchronous Asynchronous Receive Transmit
- MSSP – Master Synchronous Serial Port
- CAN – Controller Area Network

Table 10.4: Summary of Serial Communication Modules & pins

Device / Serial Communication Module	USART1 Present	USART1 Transmit / Receive Pins	USART2 Present	USART2 Transmit / Receive Pins	MSSP		CAN
					I2C	SPI	
USART Asynchronous Communication							
PIC18F458	Y	TX/RX	N		Y	Y	Y
PIC18C801	Y	TX/RX	N		Y	Y	
PIC18F8720	Y	TX1/RX1	Y	TX2/RX2	Y	Y	
USART Synchronous Communication							
PIC18F458	Y	CK/DT	N		Y	Y	Y
PIC18C801	Y	CK/DT	N		Y	Y	
PIC18F8720	Y	CK1/DT1	Y	CK2/DT2	Y	Y	

The first serial communication module is the USART that provides synchronous and asynchronous communication that is present in all three devices. The second serial communication module is the MSSP that provides SPI – Serial Peripheral Interface and I2C – Inter Integrated Circuit protocols, which is also present in all the three devices. The third serial communication module is the CAN – Controller Area Network which is present only in PIC18F458 which is based on Bosch CAN2 specification. The MSSP & CAN modules are presented in serial protocols section of chapter 15.

The introduction to serial communication has been presented in section 7.3. It includes description of asynchronous and synchronous communication. The RS232 standard is presented in Section 7.4 and voltage level converters for RS232 to TTL & CMOS in section 7.5. The description given henceforth builds up on those topics discussed earlier.

The USART is also known as SCI – Serial Communication Interface. It can be configured as asynchronous or synchronous communication system. The Asynchronous communication is employed in CRT terminals, Personal Computers. The Synchronous communication is employed for communication with peripheral devices such as A/D or D/A converters, Serial EEPROMs etc.

When it is in asynchronous mode the USART module can communicate in Full Duplex mode and when it is in synchronous mode the USART module can communicate in Half Duplex mode as a master or a slave. The PIC18 USART communication modes are summarised as follows.

1. Asynchronous Communication
 - Full Duplex, Transmit/Receive on TX/RX pins
2. Synchronous Communication
 - Master in Half Duplex mode, Clock/ Data Transmit-Receive on CK/DT pins
 - Slave in Half Duplex mode, Clock/ Data Transmit-Receive on CK/DT pins

The PIC18F458 and PIC18C801 have a single USART but the PIC18F8720 has 2 USARTs – USART1 and USART2. The USART1 and USART2 of PIC18F8720 are independent of each other and can have separate baud rates and frame formats. However, the transmit and receive sections of the same USART module use the single frame format, baud rate register and same baud rate for both transmit and receive and hence transmit section is not independent of receiver section.

Table 10.5: Summary of Pins associated with USART modules in different PIC18 devices

PORT/Pin Name	Pin Notation	PIC18F458	PIC18C801	PIC18F8720
		PORT Pin	PORT Pin	PORT Pin
Asynchronous Communication				
Serial Transmit 1	TX1	RC6, TX	RC6, TX	RC6, TX1
Serial Receive 1	RX1	RC7, RX	RC7, RX	RC7, RX1
Serial Transmit 2	TX2	-	-	RG1, TX2
Serial Receive 2	RX2	-	-	RG2, RX2
Synchronous Communication				
Synchronous Clock1	CK1	RC6, CK	RC6, CK	RC6, CK1
Serial Data1	DT1	RC7, DT	RC7, DT	RC7, DT1
Synchronous Clock2	CK2	-	-	RG1, CK2
Serial Data2	DT2	-	-	RG2, DT2

A summary of pins associated with USART modules in different PIC18 devices is presented in Table 10.5. The USART consists of transmit and receive lines. The pin RC6 is used for transmission – TXD line and RC7 is used for receive – RXD line for USART1 in all three devices. Hence 6th pin of PORTC pin the RC6 must be configured as output by clearing the

TRISC<6> bit and 7th pin of PORTC pin the RC7 must be configured as input by setting the TRISC<7> bit. Similarly, in PIC18F8720 which also the second USART the RG1 and RG2 are used for transmit and receive. Hence the TRISG<1> bit must be cleared to 0 to configure as transmit output pin and TRISG<2> must be set to 1 to configure as receive input pin.

The registers associated with USART are presented in Table 10.6a. Since PIC18F458 and PIC18C801 contains only one USART there is no postfix of 1 or 2 for the respective registers or associated bits in these devices. However, 2 USARTs are present in PIC18F8720 and hence all the USART registers and the corresponding bits have postfix of 1 or 2 depending on the USART1 or USART2. The names of the registers are SPBRGx, RCREGx, TXREGx, RCSTAx, TXSTAx. The corresponding bit names are SPENx, RX9x etc. for all the bits. The registers and respective bits are shown in Table 10.6b.

Table 10.6a: USART Serial Port related registers, Common to PIC18F458 and PIC18C801

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
SPBRG	Baud Rate Generator Register							
RCREG	USART Receive Register							
TXREG	USART Transmit Register							
RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D
TXSTA	CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D

Table 10.6b: USART Serial Port related registers, PIC18F8720

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
SPBRGx	Baud Rate Generator Register x							
RCREGx	USARTx Receive Register							
TXREGx	USARTx Transmit Register							
RCSTAx	SPENx	RX9x	SRENx	CRENx	ADDENx	FERRx	OERRx	RX9Dx
TXSTAx	CSRCx	TX9x	TXENx	SYNCx	—	BRGHx	TRMTx	TX9Dx

However, the library defined and used as part of MPLABx allows the name of the bits to be used without the postfix also i.e. SPEN, RX9 for RCSTA1 and RCSTA2 of both USART1 and USART2. This also applies to TXSTA register. However, it is encouraged to follow the prefix for uniformity as the registers must use the postfix.

The interrupt related flag, enable and priority bits and respective registers for PIC18F458 and PIC18C801 is shown in Table 10.7a and for PIC18F8720 is shown in Table 10.7b. The following discussion presents the use of PIC18F458 which is applicable to other devices also with the corresponding postfix.

Table 10.7a: USART Interrupt bits & registers, Common to PIC18F458 and PIC18C801

S.No	Interrupt Name	PIC18F458, PIC18C801					
		Flag	Flag Reg	Enable	Enable Reg	Priority	Priority Reg
1	USART1 Receive	RCIF	PIR1	RCIE	PIE1	RCIP	IPR1
2	USART1 Transmit	TXIF	PIR1	TXIE	PIE1	TXIP	IPR1
3	USART2 Receive	-	-	-	-	-	-
4	USART2 Transmit	-	-	-	-	-	-

Table 10.7b: USART Interrupt bits & registers, PIC18F8720

S.No	Interrupt Name	PIC18F8720					
		Flag	Flag Reg	Enable	Enable Reg	Priority	Priority Reg
1	USART1 Receive	RC1IF	PIR1	RC1IE	PIE1	RC1IP	IPR1
2	USART1 Transmit	TX1IF	PIR1	TX1IE	PIE1	TX1IP	IPR1
3	USART2 Receive	RC2IF	PIR3	RC2IE	PIE3	RC2IP	IPR3
4	USART2 Transmit	TX2IF	PIR3	TX2IE	PIE3	TX2IP	IPR3

10.3 USART Asynchronous Mode

The USART module of PIC18 is configured and controlled through two registers.

- Transmit Status and Control Register: TXSTA
- Receive Status and Control Register: RCSTA

The register formats of TXSTA & RCSTA and details of the associated bit are given below.

TXSTA: Transmit Status and Control Register

R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0	R-1	R/W-0
CSRC	TX9	TXEN	SYNC	—	BRGH	TRMT	TX9D
bit 7							bit 0

Bit No	Bit	Description
bit 7	CSRC	Clock Source Select bit <u>Asynchronous mode:</u> Don't care. <u>Synchronous mode:</u> 1 = Master mode (clock generated internally from BRG) 0 = Slave mode (clock from external source)
bit 6	TX9	9-bit Transmit Enable bit 1 = Selects 9-bit transmission 0 = Selects 8-bit transmission
bit 5	TXEN	Transmit Enable bit 1 = Transmit enabled 0 = Transmit disabled Note: SREN/CREN overrides TXEN in Sync mode.

bit 4	SYNC	USART Mode Select bit 1 = Synchronous mode 0 = Asynchronous mode
bit 3	Unimplemented	Read as '0'
bit 2	BRGH	High Baud Rate Select bit <u>Asynchronous mode:</u> 1 = High speed 0 = Low speed <u>Synchronous mode:</u> Unused in this mode.
bit 1	TRMT	Transmit Shift Register Status bit 1 = TSR empty 0 = TSR full
bit 0	TX9D	9th bit of Transmit Data Can be address/data bit or a parity bit.

RCSTA: Receive Status and Control Register

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R-0	R-0	R-x
SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D

bit 7

bit 0

Bit No	Bit	Description
bit 7	SPEN	Serial Port Enable bit 1 = Serial port enabled (configures RX/DT and TX/CK pins as serial port pins) 0 = Serial port disabled
bit 6	RX9	9-bit Receive Enable bit 1 = Selects 9-bit reception 0 = Selects 8-bit reception
bit 5	SREN	Single Receive Enable bit <u>Asynchronous mode:</u> Don't care. <u>Synchronous mode – Master:</u> 1 = Enables single receive 0 = Disables single receive (this bit is cleared after reception is complete) <u>Synchronous mode – Slave:</u> Unused in this mode.
bit 4	CREN	Continuous Receive Enable bit <u>Asynchronous mode:</u> 1 = Enables continuous receive 0 = Disables continuous receive <u>Synchronous mode:</u> 1 = Enables continuous receive until enable bit CREN is cleared (CREN overrides SREN) 0 = Disables continuous receive

bit 3	ADDE N	Address Detect Enable bit <u>Asynchronous mode 9-bit (RX9 = 1):</u> 1 = Enables address detection, enables interrupt and load of the receive buffer when RSR<8> is set 0 = Disables address detection, all bytes are received and ninth bit can be used as parity bit
bit 2	FERR	Framing Error bit 1 = Framing error (can be updated by reading RCREG register and receive next valid byte) 0 = No framing error
bit 1	OERR	Overrun Error bit 1 = Overrun error (can be cleared by clearing bit CREN) 0 = No overrun error
bit 0	RX9D	9th bit of Received Data Can be address/data bit or a parity bit.

The USART serial port of PIC18 is enabled by setting the Serial Port Enable bit – SPEN of the Receive Status and Control Register – RCSTA. The RCSTA<7> must be set to 1 to enable the serial port and configures RX/DT and TX/CK as serial port pins.

The USART is configured to be in asynchronous mode through the USART Mode Select bit - SYNC of the Transmit Status and Control Register – TXSTA. The SYNC i.e. TXSTA<4> must be cleared to 0 to configure the USART into the asynchronous mode and SYNC bit must be set to 1 for synchronous mode. The PIC18's USART when configured in asynchronous mode operates in full-duplex manner allowing data to be transmitted and received at the same time.

The USART of PIC18 devices uses NRZ – Non-Return-to-Zero format i.e. 1 Start bit, 8 or 9 Data bits, and 1 stop bit. The PIC18 USART hardware does not support parity. However, it can be implemented through software using the 9th data bit.

The transmitter and receiver of the USART are independent of one another and are double buffered with the exception that they use the same data format and baud rate. The start bit is always followed by the least significant bit. Hence the LSB is the first of the data bits that is either transmitted or received. This is however not the case in I2C where MSB is transmitted first. The PIC18 presented here does not support asynchronous data transfers during sleep as UART is stopped.

10.3.1 Baud Rate Calculation

The PIC18 provides a high baud rate select feature which is used when higher baud rates are desired. It is enabled through the High Baud Rate Select bit – BRGH. It is associated with the 2nd bit of TXSTA registers. In asynchronous mode, when it is set to 1, the high speed is selected, dividing the oscillator frequency by 16. When it is set to 0 the oscillator frequency is divided by 64 to select the low speed. A summary of the same is shown in Table 10.8.

Table 10.8 Baud Rate Formula

SYNC	Mode	BRGH = 0 (Low Speed)	BRGH = 1 (High Speed)
0	Asynchronous	Baud Rate = FOSC/ (64 (x + 1))	Baud Rate = FOSC/ (16 (x + 1))
1	Synchronous	Baud Rate = FOSC/ (4 (x + 1))	NA

Note: x is the value of SPBRG in the range of 0 to 255.

Table 10.9 Baud Rates for Asynchronous Mode: BRGH = 0

Crystal Frequency, MHz	1200			2400			4800		
	SPBRG Value, Dec	Rate	Error %	SPBRG Value, Dec	Rate	Error %	SPBRG Value, Dec	Rate	Error %
1	12	1201.92	0.1603	6	2232.14	-6.9940	2	5208.33	8.5069
4	51	1201.92	0.1603	25	2403.85	0.1603	12	4807.69	0.1603
5.0688	65	1200	0.0000	32	2400	0.0000	16	4658.82	-2.9412
10	129	1201.92	0.1603	64	2403.85	0.1603	32	4734.85	-1.3573
11.0592	143	1200	0.0000	71	2400	0.0000	35	4800	0.0000
12	155	1201.92	0.1603	77	2403.85	0.1603	38	4807.69	0.1603
16	207	1201.92	0.1603	103	2403.85	0.1603	51	4807.69	0.1603
20	259	1201.92	0.1603	129	2403.85	0.1603	64	4807.69	0.1603
22	285	1201.92	0.1603	142	2403.85	0.1603	71	4774.31	-0.5353
25	325	1198.24	-0.1470	162	2396.47	-0.1470	80	4822.53	0.4694
33	429	1199.13	-0.0727	214	2398.26	-0.0727	106	4818.93	0.3943
40	520	1199.62	-0.0320	259	2403.85	0.1603	129	4807.69	0.1603

Crystal Frequency, MHz	9600			19200			Min Rate	Max Rate
	SPBRG Value, Dec	Rate	Error %	SPBRG Value, Dec	Rate	Error %	SPBRG = 255	SPBRG = 0
1	1	7812.5	-18.6198	0	15625	-18.6198	61.04	15625
4	6	8928.57	-6.9940	2	20833.33	8.5069	244.14	62500
5.0688	7	9900	3.1250	3	19800	3.1250	309.38	79200
10	15	9765.63	1.7253	7	19531.25	1.7253	610.35	156250
11.0592	17	9600	0.0000	8	19200	0.0000	675	172800
12	19	9375	-2.3438	9	18750	-2.3438	732.42	187500
16	25	9615.38	0.1603	12	19230.77	0.1603	976.56	250000
20	32	9469.7	-1.3573	15	19531.25	1.7253	1220.7	312500
22	35	9548.61	-0.5353	17	19097.22	-0.5353	1342.77	343750
25	40	9527.44	-0.7558	19	19531.25	1.7253	1525.88	390625
33	53	9548.61	-0.5353	26	19097.22	-0.5353	2014.16	515625
40	64	9615.38	0.1603	32	18939.39	-1.3573	2441.41	625000

The expression for desired Baud Rate is given as

$$B.R = F_{Osc} / (64 (x + 1))$$

Where F_{Osc} is the frequency of the crystal oscillator and

x represents the Serial Port Baud Rate Generator register - SPBRG

The value of SPBRG is found as

$$x = ((F_{Osc} / B.R) / 64) - 1$$

Table 10.10 Baud Rates for Asynchronous Mode: BRGH = 1

Crystal Frequency, MHz	1200			2400			4800		
	SPBRG Value, Dec	Rate	Error %	SPBRG Value, Dec	Rate	Error %	SPBRG Value, Dec	Rate	Error %
1	51	1201.92	0.1603	25	2403.85	0.1603	12	4807.69	0.1603
4	207	1201.92	0.1603	103	2403.85	0.1603	51	4807.69	0.1603
5.0688	263	1200	0	131	2400	0	65	4800	0
10	520	1199.62	-0.032	259	2403.85	0.1603	129	4807.69	0.1603
11.0592	575	1200	0	287	2400	0	143	4800	0
12	624	1200	0	312	2396.17	-0.1597	155	4807.69	0.1603
16	832	1200.48	0.04	416	2398.08	-0.0799	207	4807.69	0.1603
20	1041	1199.62	-0.032	520	2399.23	-0.032	259	4807.69	0.1603
22	1145	1199.83	-0.0145	572	2399.65	-0.0145	285	4807.69	0.1603
25	1301	1200.08	0.0064	650	2400.15	0.0064	325	4792.94	-0.147
33	1718	1199.83	-0.0145	858	2401.05	0.0437	429	4796.51	-0.0727
40	2082	1200.19	0.016	1041	2399.23	-0.032	520	4798.46	-0.032

Crystal Frequency, MHz	9600			19200			Min Rate	Max Rate
	SPBRG Value, Dec	Rate	Error %	SPBRG Value, Dec	Rate	Error %	SPBRG = 255	SPBRG = 0
1	6	8928.57	-6.994	2	20833.33	8.5069	244.14	62500
4	25	9615.38	0.1603	12	19230.77	0.1603	976.56	250000
5.0688	32	9600	0	16	18635.29	-2.9412	1237.5	316800
10	64	9615.38	0.1603	32	18939.39	-1.3573	2441.41	625000
11.0592	71	9600	0	35	19200	0	2700	691200
12	77	9615.38	0.1603	38	19230.77	0.1603	2929.69	750000
16	103	9615.38	0.1603	51	19230.77	0.1603	3906.25	1000000
20	129	9615.38	0.1603	64	19230.77	0.1603	4882.81	1250000
22	142	9615.38	0.1603	71	19097.22	-0.5353	5371.09	1375000
25	162	9585.89	-0.147	80	19290.12	0.4694	6103.52	1562500
33	214	9593.02	-0.0727	106	19275.7	0.3943	8056.64	2062500
40	259	9615.38	0.1603	129	19230.77	0.1603	9765.63	2500000

Calculating Baud Rate Error

The difference between the required baud rate and generated baud rate due to the used SPBRG value is termed as baud rate error.

For Baud Rate of 9600

$$x = ((16000000/9600)/64) - 1 = 25.042 \approx 25$$

For an x of 25 the achieved Baud Rate is given as

$$B.R_{ach} = 16000000 / (64 (25 + 1)) = 9615$$

The Error e is given as difference between the required and achieved Baud Rate

$$e = \text{Calculated Baud Rate} - \text{Desired Baud Rate} = 9615 - 9600 = 15$$

The Relative error percentage is given as

$$e_r \% = ((9615 - 9600) / 9600) * 100 = (15/9600) * 100 = 0.16\%$$

The baud rates, SPBRG values for different clock frequencies in low speed i.e. BRGH = 0 and high speed BRGH=1 is given in Table 10.9 and Table 10.10. The PIC18F458, PIC18F8720 in microcontroller mode could use crystal frequency up to 40MHz while the PIC18C801 and PIC18F8720 in other modes could use crystals up to a frequency of only 25MHz. Hence frequencies up to 40MHz are presented here.

10.3.2 Asynchronous Transmit

While the asynchronous mode is selected by setting the SYNC bit - TXSTA<4> to a 0, the asynchronous data transmission is controlled by a Transmit Enable bit – TXEN. The TXEN i.e. TXSTA<5> bit must be set to enable the data transmission. The block diagram of asynchronous transmission is shown in fig 10.5

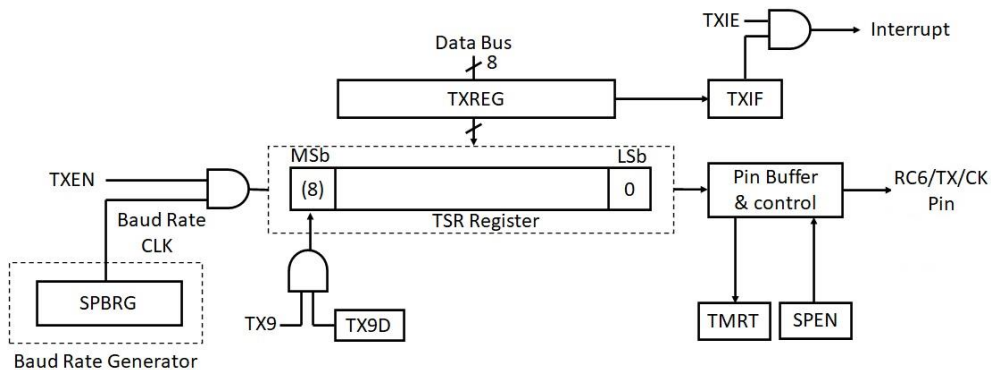


Fig 10.5 USART Transmit Block diagram

The transmitter section of the USART is double buffered indicating that it contains two buffers or registers:

- Holding register – TXREG
- Shifting register – TSR

Transmit Register – TXREG:

The Transmit Register – TXREG acts as a holding register which is accessible to the programmer. This register is readable and writeable. The value that must be transmitted

serially should be loaded into TXREG for it to be transmitted. The value of TXREG is transferred into the TSR register for shifting.

The Transmit Shift Register – TSR:

The Transmit Shift Register – TSR is the shifting register which operates as a parallel in, and serial out register. This register is not mapped in RAM and hence it is not accessible to the user. The data that is loaded into the TXREG is transferred into the TSR register for shifting. The TXREG is loaded into the TSR only after the transmission of stop bit related to previous data byte. The data in TSR register is serially shifted on to the TXD/RC6 pin.

When the TXREG value is transferred into the TSR register the Transmit Interrupt Flag - TXIF is set. The TXIF flag is set irrespective of interrupt enable. The setting of TXIF flag generates an interrupt if the Transmit Interrupt is enabled along with the required global interrupts and respective IPEN setting and priority interrupt enable. The transmit interrupt is enabled by setting the Transmit Interrupt Enable bit – TXIE to a 1. The TXIF flag bit is associated with PIR1 register and TXIE bit is associated with PIE1 register. The TXIF flag can be cleared or reset by loading a value into TXREG and cannot be cleared in software through bit/byte clear instructions. The TXIF flag is set when the transmitter is enabled by setting the TXEN bit of TXSTA<5> register to 1.

The TXIF flag indicates that the TXREG is empty whereas TRMT bit of TXSTA<1> register indicates that the TSR register is empty. The TRMT bit is a read only bit. It is set when the TSR register is empty and is set upon power up as TSR is empty. The sample waveform of back-to-back asynchronous transmission is shown in fig 10.6.

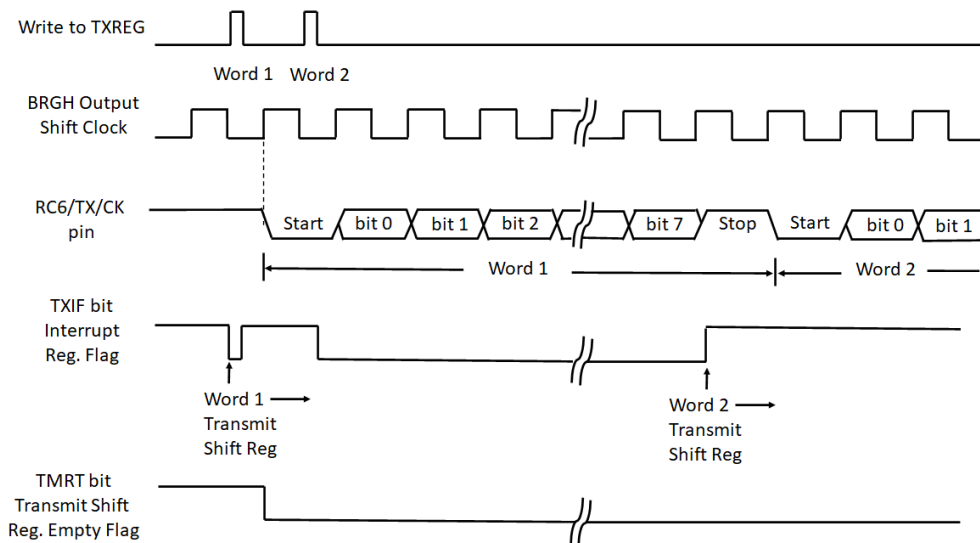


Fig 10.6 USART Asynchronous Transmit Waveform

The TSR register is shifted right by one bit for every clock pulse. The least significant bit is transmitted first among the data bits. The frequency of the pulse is dependent on the required baud rate and is configured through the Serial Port Baud Rate Generator register - SPBRG. The baud rate configuration allows for a low speed or high speed i.e. x16 or x64 of the bit shift rate. The high speed is selected when the High Baud Rate Select bit – BRGH of TXSTA<2> register is set to 1; else low speed is selected. The baud rate configuration is discussed later in this section. Though the transmission and reception are independent, the baud rate generation and asynchronous data format is common for transmission and reception.

A multi-master configuration requires a control bit for communication. Hence PIC18's USART is provided with a 9th bit for multi-processor communication. The 9th bit is enabled through the 9th bit Transmit Enable bit - TX9 of TXSTA<6> register. The least significant bit of the transmit status and control register TXSTA<0> acts as the actual 9th data bit – TX9D. If a 1 is to be transmitted as the 9th bit, the TX9D i.e. TXSTA<0> must be set; else it must be cleared to 0.

Operation sequence steps for asynchronous transmission:

- Load the SPBRG register with the appropriate value to select the required baud rate.
- If high speed baud rate generation is required, set the BRGH bit of TXSTA<2> register to a 1.
- The 6th pin of PORTC pin, the RC6 must be configured as output for data transmission by clearing the TRISC<6> bit
- Enable the serial port by setting the SPEN bit of RCSTA<7> register to a 1.
- Select asynchronous communication by setting the SYNC bit - TXSTA<4> to a 0.
- If interrupt is desired, Enable the transmit interrupt by setting the TXIE bit of PIE1 register to a 1.
- If multiprocessor communication or software parity is desired, set the 9th bit transmit enable bit - TX9 of TXSTA<6> bit to a 1. Can be used as address/data bit.
- Enable the serial transmission by setting the TXEN bit of TXSTA<5> register to a 1. This also sets the TXIF flag and loading data into TXREG clears it.
- If 9th bit transmission is selected, load the parity/communication bit into TX9D bit of TXSTA<0> register.
- Load the TXREG register with the data that must be serially transmitted. After the data is loaded into the TXREG, the TXIF is only cleared after the 2nd instruction cycle.

Program 10.3: Transmit the string “PIC” serially using a crystal oscillator frequency of 11.0592 MHz at 9600 baud rate.

```

#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR
ORG      0x00          ; Set Origin 0x0000 of Flash memory
GOTO     START         ; Go to Start of Main program.
ORG      0x0100        ; Set Origin 0x0100 for Main Program
START    BCF           TRISC, TX      ; TRISC<6> = 0, Set RC6 as Output for TX
                                           ; functionality
        MOVLW         0x11          ; WREG = 0x11
        MOVWF         SPBRG        ; SPBRG = WREG = 0x11, Set Baud Rate
                                           ; 9600 at 11.0592MHz 1 Start, 8 Data,
                                           ; 1 Stop Bits.
        BSF           RCSTA,SPEN    ; Enable Serial Port. RCSTA<7> = 1
        MOVLW         0x20          ; WREG = 0x20
        MOVWF         TXSTA        ; TXSTA = 0x20, Set Asynchronous Mode,
                                           ; Enable serial transmission
HERE     MOVLW         'P'          ; WREG = 'P' = 0x50
        MOVWF         TXREG        ; TXREG = WREG = 0x50
L1       BTFSS         PIR1, TXIF    ; Check if the TXIF flag is set, i.e. TXREG is
                                           ; empty, Skip next instruction if empty
                                           ; i.e. TXIF is set
        BRA           L1           ; Branch to L1 if TXIF is 0 or TXREG is not
                                           ; loaded to TSR register.
        MOVLW         'I'          ; WREG = 'I' = 0x49
        MOVWF         TXREG        ; TXREG = WREG = 0x49
L2       BTFSS         PIR1, TXIF    ; Check if the TXIF flag is set, i.e. TXREG is
                                           ; empty, Skip next instruction if empty i.e.
                                           ; TXIF is set
        BRA           L2           ; Branch to L2 if TXIF is 0 or TXREG is not
                                           ; loaded to TSR register.
        MOVLW         'C'          ; WREG = 'C' = 0x43
        MOVWF         TXREG        ; TXREG = WREG = 0x43
L3       BTFSS         PIR1, TXIF    ; Check if the TXIF flag is set, i.e. TXREG is
                                           ; empty, Skip next instruction if empty i.e.
                                           ; TXIF is set
        BRA           L3           ; Branch to L3 if TXIF is 0 or TXREG is not
                                           ; loaded to TSR register.
        BRA           HERE        ; Branch to Here
        END              ; End

```

Program 10.4: Configure PIC18 USART in asynchronous mode to transfer data serially. Transfer the data that is available from file location starting from 0x30 till a 0 value is encountered. Assume crystal oscillator frequency of 11.0592 MHz to transfer at 9600 baud rate with sub routine.

```

#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR
ORG      0x00           ; Set Origin 0x0000 of Flash memory
GOTO     START          ; Go to Start of Main program.
ORG      0x0100         ; Set Origin 0x0100 for Main Program
START    BCF            TRISC, TX      ; TRISC<6> = 0, Set RC6 as Output for TX
                                           ; functionality
        MOVLW          0x11           ; WREG = 0x11
        MOVWF          SPBRG          ; SPBRG = WREG = 0x11, Set Baud Rate
                                           ; 9600 at 11.0592MHz 1 Start, 8 Data, 1
                                           ; Stop Bits.
        BSF            RCSTA,SPEN     ; Enable Serial Port. RCSTA<7> = 1
        MOVLW          0x20           ; WREG = 0x20
        MOVWF          TXSTA          ; TXSTA = 0x20, Set Asynchronous Mode,
                                           ; Enable serial transmission
        LFSR           0,0x30         ; FSR0 = 0x30
HERE     MOVF           POSTINC0,0,0   ; WREG = [FSR0] = [0x30], FSR0 = FSR0 + 1
        TSTFSZ         WREG,0         ; Check if the WREG is 0, i.e. WREG is
                                           ; empty, Skip next instruction if empty
        BRA            CX             ; Branch to CX if WREG is not 0
        BRA            ST             ; Branch to ST, END if WREG is 0
CX       CALL           M_TX           ; Call M_TX if WREG is not 0
        BRA            HERE          ; Branch to Here, Repeat Serial
                                           ; Transmission
M_TX     NOP             ; No Operation
L1       BTFSS         PIR1, TXIF     ; Check if the TXIF flag is set, i.e. TXREG is
                                           ; empty, Skip next instruction if empty i.e.
                                           ; TXIF is set
        BRA            L1             ; Branch to L3 if TXIF is 0 or TXREG is not
                                           ; loaded to TSR register.
        MOVWF          TXREG          ; TXREG = WREG = 0x43
        RETURN          ; Return from M_TX module
ST       NOP             ; No Operation
        END              ; End

```

Program 10.5: Configure PIC18 USART in asynchronous mode to transfer data serially. Transfer the data that is available from file location starting from 0x30 till a 0 value is encountered. Assume crystal oscillator frequency of 11.0592 MHz to transfer at 9600 baud rate using interrupts.

```

#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR
ORG      0x00           ; Set Origin 0x0000 of Flash memory
GOTO     START          ; Go to Start the Main program.
ORG      0x08           ; Set Origin 0x0008 of High Priority

```

			; Interrupt
	BTFSF	PIR1,TXIF	; Check if interrupt is due to Transmit
	RETFIE		; interrupt flag
	GOTO	TX_ISR	; If not due to TXIF return from ISR
			; If interrupt is due to Transmit interrupt,
			; go to TX_ISR
	ORG	0x50	; Set Origin 0x0050 of TX_ISR
TX_ISR	MOVF	POSTINC0,0,0	; WREG = [FSR0] = [0x31], FSR0 = FSR0 + 1
	TSTFSZ	WREG,0	; Check if the WREG is 0, i.e. WREG is
			; empty, Skip next instruction if empty
	BRA	TS	; Branch to TS, if WREG is not 0
	BRA	ST	; Branch to ST, if WREG is 0
TS	MOVWF	TXREG	; TXREG = WREG, Clears TXIF flag after 2 nd
			; instruction cycle
	NOP		; No Operation
	RETFIE		; Return from M_TX module
ST	BCF	PIE1,TXIE	; Disable transmit interrupt.
	RETFIE		; Return from M_TX module
	ORG	0x0100	; Set Origin 0x0100 for Main Program
START	BCF	TRISC, TX	; TRISC<6> = 0, Set RC6 as Output for TX
			; functionality
	MOVLW	0x11	; WREG = 0x11
	MOVWF	SPBRG	; SPBRG = WREG = 0x11, Set Baud Rate
			; 9600 at 11.0592MHz 1 Start, 8 Data, 1
			; Stop Bits.
	BSF	RCSTA,SPEN	; Enable Serial Port. RCSTA<7> = 1
	MOVLW	0x20	; WREG = 0x20
	MOVWF	TXSTA	; TXSTA = 0x20, Set Asynchronous Mode,
			; Enable serial transmission
	LFSR	0,0x30	; FSR0 = 0x30
	BSF	PIE1,TXIE	; Enable Transmit Interrupt
	BSF	INTCON,PEIE	; Enable Peripheral Interrupt
	BSF	RCON,IPEN	; Enable Priorities
	BSF	IPR1,TXIP	; Set Transmit interrupt to high priority.
	BSF	INTCON,GIE	; Enable Global Interrupt
HERE	NOP		; No Operation, Alternate function
	BRA	HERE	; Branch to Here
	END		; End

10.3.3 Asynchronous Receive

While the asynchronous mode is selected by setting the SYNC bit - TXSTA<4> to a 0, the asynchronous data receive is controlled through the Continuous Receive Enable bit – CREN. The CREN bit is associated with RCSTA<4> and it must be set to 1 to enable the reception of data. In asynchronous mode, since the data is discontinuous, the USART must be enabled in continuous receive mode and the SREN – Single Receive Enable is neglected.

The block diagram of asynchronous transmission is shown in fig 10.7. The receiver section of USART is also double buffered & hence contains 2 buffers or registers:

- Holding register – RCREG
- Shifting register – RSR

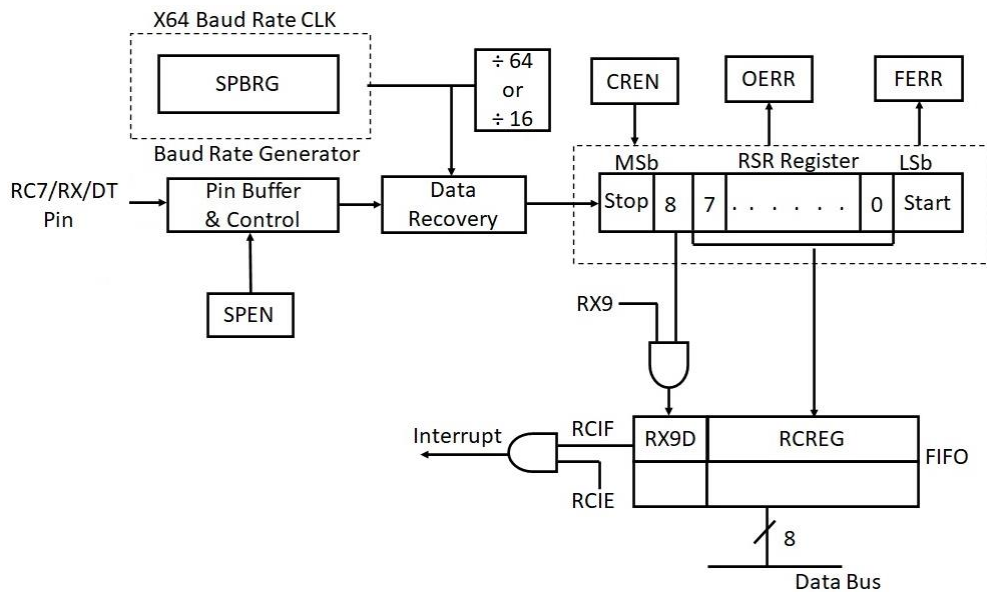


Fig 10.7 USART Receive Block diagram

Receive Register – RCREG:

The Receive Register – RCREG acts as a holding register which is accessible to the programmer. The data byte that is received serially into RSR is transferred into the RCREG for it to be read by the user.

The Receive Shift Register – RSR:

The Receive Shift Register – RSR is the shifting register which operates as a serial in and parallel out register. This register is not mapped in RAM and hence it is not accessible to the user. The incoming data is serially received on the RXD/RC7 and is stored into RSR register and shifted right every baud rate cycle. When the data byte reception is complete, the RSR register data is transferred to RCREG for it to be read by the user.

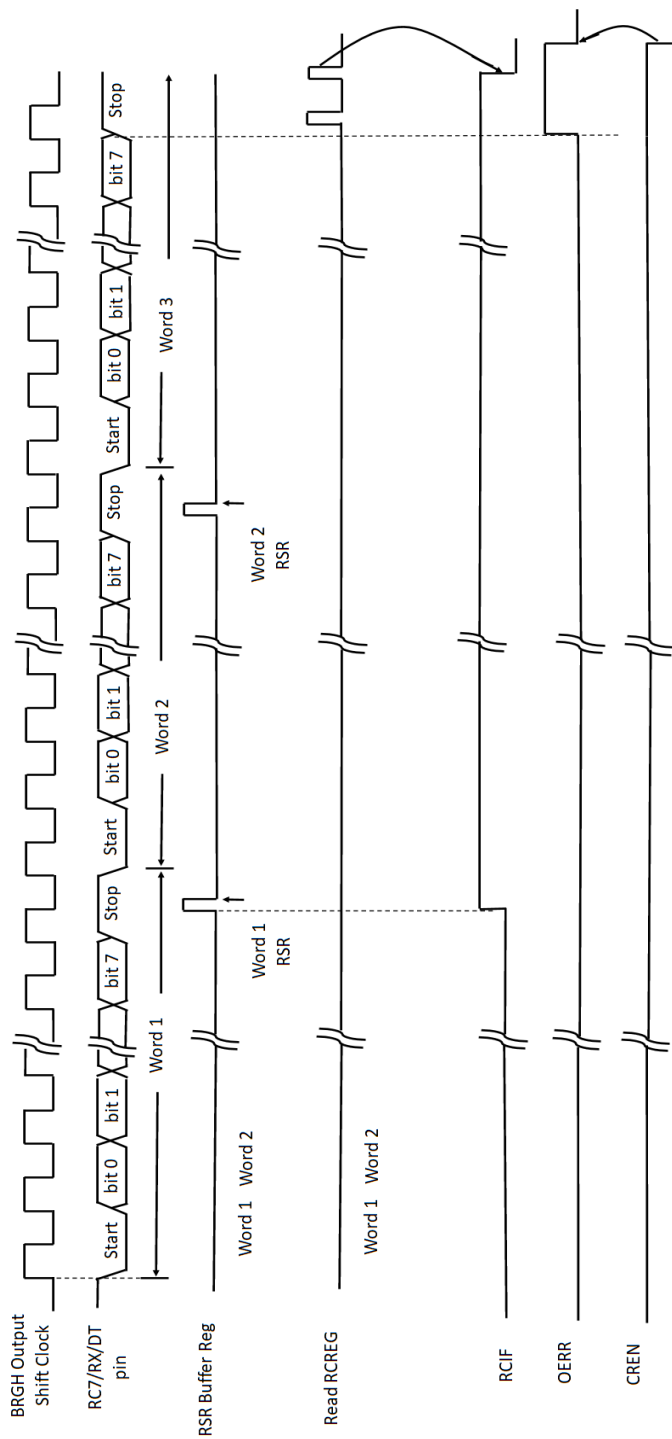


Fig 10.8 USART Asynchronous Receive Waveform

When the data reception is complete, RSR register value is transferred to RCREG and the Receive Interrupt Flag - RCIF is set. The RCIF flag indicates that the RCREG is full and must be read by the user. The RCIF flag is set irrespective of interrupt enable.

An interrupt is generated only if the RCIF flag is set and receive Interrupt is enabled along with the required global interrupts and appropriate individual priority enable for the global IPEN priority setting. The receive interrupt is enabled by setting the Receive Interrupt Enable bit – RCIE to a 1.

The RCIF flag bit is associated with PIR1 register & RCIE bit is associated with PIE1 register. The RCIF flag can be cleared or reset by reading the RCREG and cannot be cleared in software by bit/byte instructions. The sample waveform of back-to-back asynchronous reception is shown in fig 10.8.

The RSR register is shifted right by one bit for every clock pulse. The least significant bit is received first among the data bits. The frequency of the pulse is dependent on the required baud rate and is configured through the Serial Port Baud Rate Generator register - SPBRG. The baud rate configuration allows for a low speed or high speed i.e. x16 or x64 of the bit shift rate. The high speed is selected when the High Baud Rate Select bit – BRGH of TXSTA<2> register is set to 1; else low speed is selected. The baud rate configuration is discussed later in this section. Though the transmission and reception are independent, the baud rate generation and asynchronous data format is common for transmission and reception.

A multi-master configuration requires a control bit for communication. Hence PIC18's USART is provided with a 9th bit for multi-processor communication. The 9th bit is enabled through the 9th bit Receive Enable bit - RX9 of RCSTA<6> register. The received 9th bit data is stored in least significant bit of the receive status and control register RCSTA<0>. The RCSTA<0> value of 1 indicates that the received 9th bit is 1; else it is 0.

While receiving data in continuous receive mode, if the RCREGx value is not read in time, it may be overwritten by a new received data byte. This causes an Overrun Error and sets the Overrun Error bit – OERR of RCSTA<1> register to 1. The occurrence of error due to missed RCREG read causing an OERR is shown in fig 10.8. The OERR can be cleared by clearing continuous receive enable bit – CREN. Similarly, if all the bits of the frame are not received (ex: missing start, stop bit etc.) or are not in the required order, it causes a Framing Error and set the Framing Error bit – FERR of RCSTA<2> register to 1. This can be cleared by reading RCREG and after receipt of the next valid byte.

The PIC18's USART is different from a traditional USART as it is an addressable USART. It enables address detection, interrupt and load the receive buffer when RSR<8> register is set.

Operation sequence steps for asynchronous reception:

- Load the SPBRG register with the appropriate value to select the required baud rate.
- If high speed baud rate generation is required, set the BRGH bit of TXSTA<2> register to a 1.
- The 7th pin of PORTC pin the RC7 must be configured as input by setting the TRISC<7> bit.
- Enable the serial port by setting the SPEN bit of RCSTA<7> register to a 1.
- Select asynchronous communication by setting the SYNC bit - TXSTA<4> to a 0.
- If interrupt is desired, enable the receive interrupt by setting the RCIE bit of PIE1 register to a 1.
- If multiprocessor communication or software parity is desired, set the 9th bit receive enable bit - RX9 of RCSTA<>6 bit to a 1.
- Enable the serial reception by setting the CREN bit of RCSTA<4> register to a 1. The reading of RCREG register clears it.
- If data reception is complete, RCIF flag is set and an interrupt is generated if enable bit RCIE IN PIR1 register is set.
- If 9th bit reception is selected, Read the RCSTA register by reading the RC9D bit of RCSTA<0> register. When RCSTA register is read also determine if framing or overrun error has occurred.
- Read the RCREG register if no errors. If any error occurred, clear the CREN bit RCSTA<4> register.

Program 10.6: Configure PIC18 USART in asynchronous mode to receive data serially till a 0 value is encountered, store the data into file location starting from 0x30. Assume crystal oscillator frequency of 11.0592 MHz to transfer at 9600 baud rate using sub routine.

```

#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR
ORG      0x00           ; Set Origin 0x0000 of Flash memory
GOTO     START          ; Go to Start the Main program.
ORG      0x0100         ; Set Origin 0x0100 for Main Program
START    BSF            TRISC, RX      ; TRISC<7> = 1, Set RC7 as Input for RX
                                         ; functionality
          MOVLW          0x11          ; WREG = 0x11
          MOVWF          SPBRG         ; SPBRG = WREG = 0x11, Set Baud Rate 9600
                                         ; at 11.0592MHz 1 Start, 8 Data, 1 Stop Bits.
          BSF            RCSTA,SPEN    ; Enable Serial Port. RCSTA<7> = 1
          MOVLW          0x00          ; WREG = 0x00
          MOVWF          TXSTA         ; TXSTA = 0x00, Set Asynchronous Mode
HERE     BSF            RCSTA,CREN     ; Enable Continuous Receive
L1       BTFSS          PIR1, RCIF     ; Check if the RCIF flag is set, i.e Data
                                         ; received? Skip next instruction if data full

```

			; i.e. RCIF is set
	BRA	L1	; Branch to L1 if RCIF is 0 or data not
			; received, RCREG is not loaded with RSR
			; register.
	MOVWF	RCREG,0,0	; WREG = RCREG, Copy received serial data
	MOVWF	0x30,0	; [0x30] = WREG
L2	BTFSS	PIR1, RCIF	; Check if the RCIF flag is set, i.e. Data
			; received? Skip next instruction if data full
			; i.e. RCIF is set
	BRA	L2	; Branch to L2 if RCIF is 0 or data not
			; received, RCREG is not loaded with RSR
			; register.
	MOVWF	RCREG,0,0	; WREG = RCREG, Copy received serial data
	MOVWF	0x31,0	; [0x31] = WREG
L3	BTFSS	PIR1, RCIF	; Check if the RCIF flag is set, i.e. Data
			; received? Skip next instruction if data full
			; i.e. RCIF is set
	BRA	L3	; Branch to L1 if RCIF is 0 or data not
			; received, RCREG is not loaded with RSR
			; register.
	MOVWF	RCREG,0,0	; WREG = RCREG, Copy received serial data
	MOVWF	0x32,0	; [0x32] = WREG
	BRA	HERE	; Branch to Here
	END		; End

Program 10.7: Configure PIC18 USART in asynchronous mode to receive data serially till a 0 value is encountered, store the data into file location starting from 0x30. Assume crystal oscillator frequency of 11.0592 MHz to transfer at 9600 baud rate using interrupts.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x08	; Set Origin 0x0008 of High Priority
			; Interrupt
	BTFSS	PIR1,RCIF	; Check if interrupt is due to Receive
			; interrupt flag
	RETFIE		; If not due to RCIF return from ISR
	GOTO	RC_ISR	; If interrupt is due to Receive interrupt, go
			; to RC_ISR
	ORG	0x50	; Set Origin 0x0050 of RC_ISR
RC_ISR	MOVWF	RCREG,0,0	; WREG = RCREG
	TSTFSZ	WREG,0	; Check if the WREG is 0, i.e. WREG is
			; empty, Skip next instruction if empty
	BRA	RS	; Branch to RS, if WREG is not 0

RS	BRA	SR	; Branch to SR, if WREG is 0
	MOVWF	POSTINC0,0	; [FSR0] = WREG, FSR0 = FSR0 + 1
	NOP		; No Operation
SR	RETFIE		; Return from RC_ISR
	BCF	PIE1,RCIE	; Disable Receive interrupt.
	RETFIE		; Return from RC_ISR
START	ORG	0x0100	; Set Origin 0x0100 for Main Program
	BSF	TRISC, RX	; TRISC<7> = 1, Set RC7 as Input for RX
			; functionality
	MOVLW	0x11H	; WREG = 0x11
	MOVWF	SPBRG	; SPBRG = WREG = 0x11, Set Baud Rate
			; 9600 at 11.0592MHz 1 Start, 8 Data,
			; 1 Stop Bits.
	BSF	RCSTA,SPEN	; Enable Serial Port. RCSTA<7> = 1
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TXSTA	; TXSTA = 0x00, Set Asynchronous Mode
	BSF	RCSTA,CREN	; Enable Continuous Receive
	LFSR	0,0x30	; FSR0 = 0x30
	BSF	PIE1,RCIE	; Enable Receive Interrupt
	BSF	INTCON,PEIE	; Enable Peripheral Interrupt
	BSF	RCON,IPEN	; Enable Priorities
	BSF	IPR1,RCIP	; Set Receive interrupt to high priority.
	BSF	INTCON,GIE	; Enable Global Interrupt
HERE	NOP		; No Operation, Alternate function
	BRA	HERE	; Branch to Here
	END		; End

The synchronous mode always uses a default division of 4, hence high speed and low speed select is not available and the value of BRGH value is not applicable.

10.4 USART Synchronous Mode

The PIC18's USART can be configured to operate in Synchronous mode or Asynchronous Mode. The basic concept of synchronous mode is presented in sect 7.3 and its interfacing is shown in fig 7.9.

While the asynchronous mode is selected by setting the SYNC bit - TXSTA<4> to a 0, the synchronous mode is selected by setting the SYNC bit to a 1. The PIC18's USART operates in Half Duplex mode when it is configured in synchronous mode. This allows for data to be either transmitted or received but not at the same time. The data is transmitted on RC7/RX/DT pin while the clock is transmitted on RC6/TX/CK pin.

The structure of both the asynchronous and synchronous modes is mostly similar. However, the frame format in synchronous mode is different from asynchronous mode. While the asynchronous mode uses a start and stop bit to indicate the start and end of the frame that are an overhead, no such start or stop bits exist in synchronous mode. It is shown through the synchronous transmission of fig 10.5.1 and synchronous reception of fig 10.5.2.

In synchronous mode the device could be made to operate as a Master or as a Slave. The device could be configured as a master by setting the Clock Source Select bit – CSRC: TXSTA<7> to a 1 and set as slave when the bit is set to 0.

When it is set in master mode the clock is generated through the master's internal baud rate generator circuit and is controlled through the SPBRG register. When PIC18 is set in slave mode the clock must be generated by the external master or a separate external clock source and is not controlled by the SPBRG register.

The transmission is enabled through the Transmit Enable bit – TXEN, i.e. TXSTA<5> register and the reception is enabled using the Single Receive Enable bit – SREN: RCSTA<5> or a Continuous Receive Enable bit – CREN: RCSTA<4>.

10.4.1 Synchronous Transmit

The internal block diagram of USART transmit section in synchronous mode is identical to that of asynchronous mode as shown in fig 10.5. The transmit enable is also identical to that of asynchronous mode where TXEN: TXSTA<5> bit must be set to 1 to enable transmission in synchronous mode. As discussed earlier, in synchronous mode the data can be transmitted by a master or a slave.

For synchronous data transmission to succeed, two possible scenarios exist. They are shown in fig 10.9a & 10.9b.

- i. One device must be configured as master transmitter that generates the clock, while the second device must be configured as the slave receiver transferring data based on the master's clock.
- ii. Both devices may be configured as slaves with one as a transmitter and other as a receiver. Such a setup requires an external clock generator to generate the clock.

Master Transmit:

When the Clock Source Select bit – CSRC: TXSTA<7> is set to 1, the device is configured to be a master. The master generates the synchronous transmission clock on RC6/TX/CK output pin which is controlled through the master's internal baud rate generator control circuit configured through its SPBRG register. The slave must synchronize itself with the master clock and once synchronized, reads the incoming data by sampling the data line - RC7/RX/DT according to the incoming clock at RC6/TX/CK.

Slave Transmit:

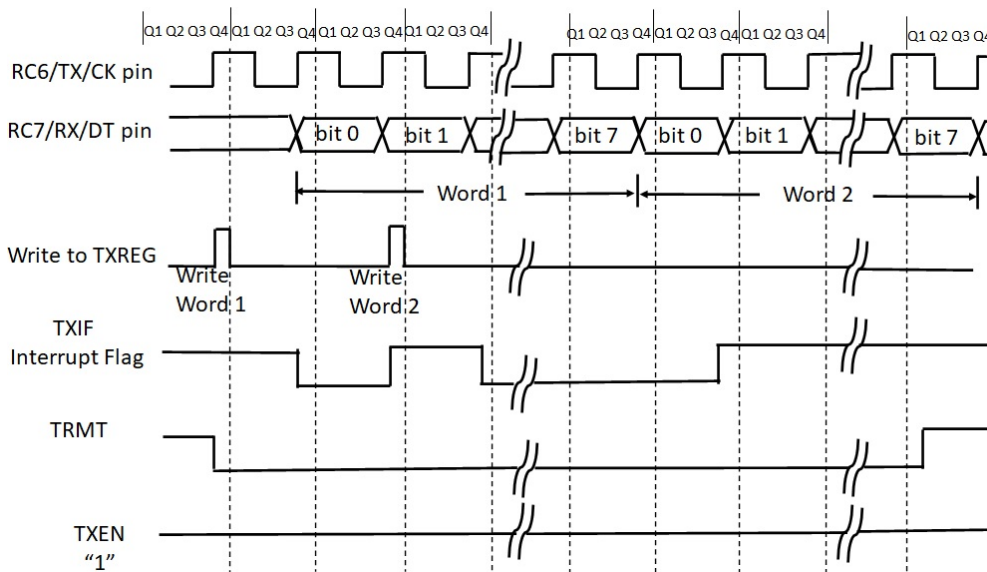
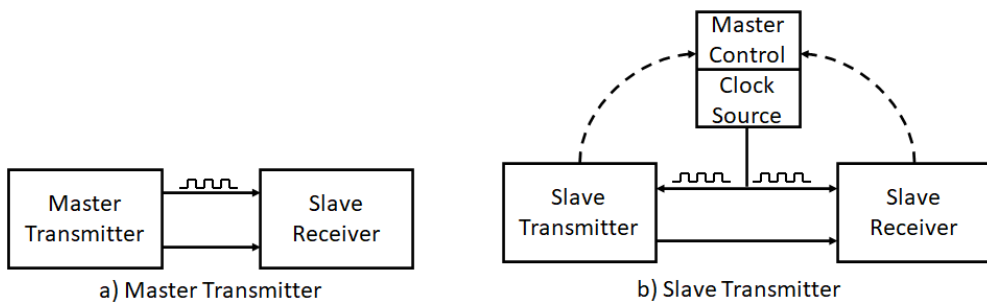
When the Clock Source Select bit – CSRC: TXSTA<7> is set to 0, the device is configured to be a slave. The RC6/TX/CK pin must now be configured as input so that the external clock is applied at this pin. The synchronous clock generated by the master or an external clock source on RC6/TX/CK pin is used by the slave to synchronize itself and to transmit the data according to the applied clock frequency.

The operation of master and slave transmit are identical except for master generating the clock and not being able to be put in sleep mode (as it generates the clock), whereas the slave can be put in sleep mode. When the available data in TXREG and TSR register is sent by slave, the TXIF flag would be set and if global enable bits and TXIE is enabled the slave can be woken from sleep mode. The transmit waveform is shown in fig 10.9c assuming RC6/TX/CK pin as output while device is a master and as input when the device is a slave.

Operation sequence steps for Synchronous Transmit – Master/Slave:

Master	Slave
Set the required baud rate by initializing the SPBRG register.	Clock is provided through external source; hence initialization of internal SPBRG register is not required.
The 6 th pin of PORTC pin - RC6 must be configured as output for clock transmission by clearing the TRISC<6> bit. The 7 th pin of PORTC pin - RC7 must be configured as output by clearing the TRISC<7> bit for data transfer.	The 6 th pin of PORTC pin - RC6 must be configured as input for clock IN by setting the TRISC<6> bit. The 7 th pin of PORTC pin - RC7 must be configured as output by clearing the TRISC<7> bit for data transfer.
Enable Synchronous mode by setting SYNC bit - TXSTA<4> to 1; Enable Serial Port by setting SPEN - RCSTA<7> to 1; Set Master mode by setting CSRC - TXSTA<7> to 1.	Enable Synchronous mode by setting SYNC bit - TXSTA<4> to 1; Enable Serial Port by setting SPEN - RCSTA<7> to 1; Set Slave mode by clearing CSRC - TXSTA<7> to 0.
If interrupts are desired, Enable TXIE bit of PIE1 register to 1	If interrupts are desired, Enable TXIE bit of PIE1 register to 1
If 9-Bit transmission is desired, set 9-bit Transmit enable TX9 bit - TXSTA<6> to 1	If 9-Bit transmission is desired, set 9-bit Transmit enable TX9 bit - TXSTA<6> to 1
Enable transmission by setting TXEN bit - TXSTA<5> to 1	Enable transmission by setting TXEN bit - TXSTA<5> to 1

If 9-bit transmission is enabled, load TX9D in TXSTA<0> with respective bit value	If 9-bit transmission is enabled, load TX9D in TXSTA<0> with respective bit value
Start transmission by loading data to TXREG. It should be noted that the TXIF flag is changed during the second instruction cycle after loading the TXREG.	Start transmission by loading data to TXREG. It should be noted that the TXIF flag is changed during the second instruction cycle after loading the TXREG.
When interrupts are desired, ensure that Global interrupts GIE & PEIE of INTCON register are set to 1 with required priority.	When interrupts are desired, ensure that Global interrupts GIE & PEIE of INTCON register are set to 1 with required priority.



c) Transmit Waveform

Fig 10.9 USART Synchronous Transmit Configurations & Waveform

Program 10.8: Configure PIC18 as a **master** that can synchronously transmit serial data from file location starting from 0x30 till a 0 value is encountered. Assume crystal oscillator frequency of 11.0592 MHz to transfer at 9600 baud rate with interrupt.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x0008 of High Priority Interrupt
	BTFSS	PIR1,TXIF	; Check if interrupt is due to Transmit interrupt
	RETFIE		; If not due to TXIF return from ISR
	GOTO	TX_ISR	; If interrupt is due to Transmit interrupt, ; go to TX_ISR
	ORG	0x50	; Set Origin 0x0050 of TX_ISR
TX_ISR	MOVF	POSTINC0,0	; WREG = [FSR0] = [0x31], FSR0 = FSR0 + 1
	TSTFSZ	WREG,0	; Check if the WREG is 0, i.e WREG is ; empty, Skip next instruction if empty
	BRA	TS	; Branch to TS, if WREG is not 0
	BRA	ST	; Branch to ST, if WREG is 0
TS	MOVWF	TXREG	; TXREG = WREG, Clears TXIF flag after 2 nd ; Instruction cycle.
	NOP		; No Operation, To Ensure 2nd instruction cycle ; time to clear TXIF
	RETFIE		; Return from M_TX module
ST	BCF	PIE1,TXIE	; Disable transmit interrupt.
	RETFIE		; Return from M_TX module
	ORG	0x0100	; Set Origin 0x0100 for Main Program
START			
; For Master RC6 is Output, TRIS<6>=0, Master Generates Clock.			
	BCF	TRISC, CK	; TRISC<6> = 0, Set RC6 as Output for CK ; functionality
	MOVLW	0x11	; WREG = 0x11
	MOVWF	SPBRG	; SPBRG = WREG = 0x11, Set Baud Rate ; 9600 at 11.0592MHz 1 Start, 8 Data, 1 ; Stop Bits.
; For Master or Slave Transmit RC7 is Output, TRIS<7>=0, Data Transmit			
	BCF	TRISC, DT	; TRISC<7> = 0, Set RC7 as Output for TX ; functionality
	BSF	RCSTA,SPEN	; Enable Serial Port. RCSTA<7> = 1
; For Master Synchronous TXSTA = 0x90			

	MOVLW	0x90	; WREG = 0x90
	MOVWF	TXSTA	; TXSTA = 0x90, Set Master Synchronous Mode
	LFSR	0,0x30	; FSR0 = 0x30
	BSF	PIE1,TXIE	; Enable Transmit Interrupt
	BSF	INTCON,PEIE	; Enable Peripheral Interrupt
	BSF	RCON,IPEN	; Enable Priorities
	BSF	IPR1,TXIP	; Set Transmit interrupt to high priority.
	BSF	TXSTA,TXEN	; Enable serial transmission
	BSF	INTCON,GIE	; Enable Global Interrupt
HERE	NOP		; No Operation, Alternate function
	BRA	HERE	; Branch to Here
	END		; End

Program 10.9: Configure PIC18 as a **slave** that can synchronously transmit serial data from file location starting from 0x30 till a 0 value is encountered. Assume crystal oscillator frequency of 11.0592 MHz to transfer at 9600 baud rate with interrupt.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x08	; Set Origin 0x0008 of High Priority Interrupt
	BTFSS	PIR1,TXIF	; Check if interrupt is due to Transmit interrupt
	RETFIE		; If not due to TXIF return from ISR
	GOTO	TX_ISR	; If interrupt is due to Transmit interrupt, ; go to TX_ISR
	ORG	0x50	; Set Origin 0x0050 of TX_ISR
TX_ISR	MOVF	POSTINC0,0,0	; WREG = [FSR0] = [0x31], FSR0 = FSR0 + 1
	TSTFSZ	WREG,0	; Check if the WREG is 0, i.e WREG is ; empty, Skip next instruction if empty
	BRA	TS	; Branch to TS, if WREG is not 0
	BRA	ST	; Branch to ST, if WREG is 0
TS	MOVWF	TXREG	; TXREG = WREG, Clears TXIF flag after 2 nd ; Instruction cycle.
	NOP		; No Operation, To Ensure 2nd instruction cycle time to clear TXIF
	RETFIE		; Return from M_TX module
ST	BCF	PIE1,TXIE	; Disable transmit interrupt.
	RETFIE		; Return from M_TX module
	ORG	0x0100	; Set Origin 0x0100 for Main Program
START			

; For Slave RC6 is Input, TRIS<6>=1, External Source Generates Clock. ; So no baud rate setting			
	BSF	TRISC, CK	; TRISC<6> = 1, Set RC6 as Input for CK ; functionality
; For Master or Slave Transmit RC7 is Output, TRIS<7>=0, Data Transmit			
	BCF	TRISC, DT	; TRISC<7> = 0, Set RC7 as Output for TX ; functionality
	BSF	RCSTA, SPEN	; Enable Serial Port. RCSTA<7> = 1
; For Slave Synchronous TXSTA = 0x10			
	MOVLW	0x10	; WREG = 0x10
	MOVWF	TXSTA	; TXSTA = 0x10, Set Slave Synchronous Mode
	LFSR	0,0x30	; FSR0 = 0x30
	BSF	PIE1, TXIE	; Enable Transmit Interrupt
	BSF	INTCON, PEIE	; Enable Peripheral Interrupt
	BSF	RCON, IPEN	; Enable Priorities
	BSF	IPR1, TXIP	; Set Transmit interrupt to high priority.
	BSF	TXSTA, TXEN	; Enable serial transmission
	BSF	INTCON, GIE	; Enable Global Interrupt
HERE	NOP		; No Operation, Alternate function
	BRA	HERE	; Branch to Here
	END		; End

10.4.2 Synchronous Receive

The internal block diagram of USART receive section in synchronous mode is identical to that of asynchronous mode as shown in fig 10.7.

While the asynchronous receive is always enabled through the Continuous Receive Enable bit – CREN: RCSTA<4> the synchronous data receive is controlled through Single Receive Enable – SREN: RCSTA<5> or Continuous Receive Enable bit – CREN: RCSTA<4>. The SREN - RCSTA<5> or the CREN - RCSTA<4> bit must be set to enable the reception of data in master mode. The SREN is neglected in slave mode.

When only a byte transfer is desired, the USART must be configured in single receive mode by setting the SREN bit - RCSTA<5> to a 1. The SREN bit is reset automatically after reception of single byte. For transfer of another byte the SREN must be set again.

When multiple byte transfers are desired, the USART must be configured in continuous receive mode by setting the CREN bit - RCSTA<4> to a 1. If both the SREN and CREN bit are set, the continuous receive mode overrides the single receive mode.

For synchronous data reception to succeed, two possible scenarios exist. They are shown in fig 10.10a & 10.10b.

- i. One device must be configured as master receiver that generates the clock, while the second device must be configured as the slave transmitter that transfers data based on the master's clock.
- ii. Both the devices may be configured as slaves with one as a transmitter and the other as a receiver. Such a setup requires an external clock generator to generate the clock.

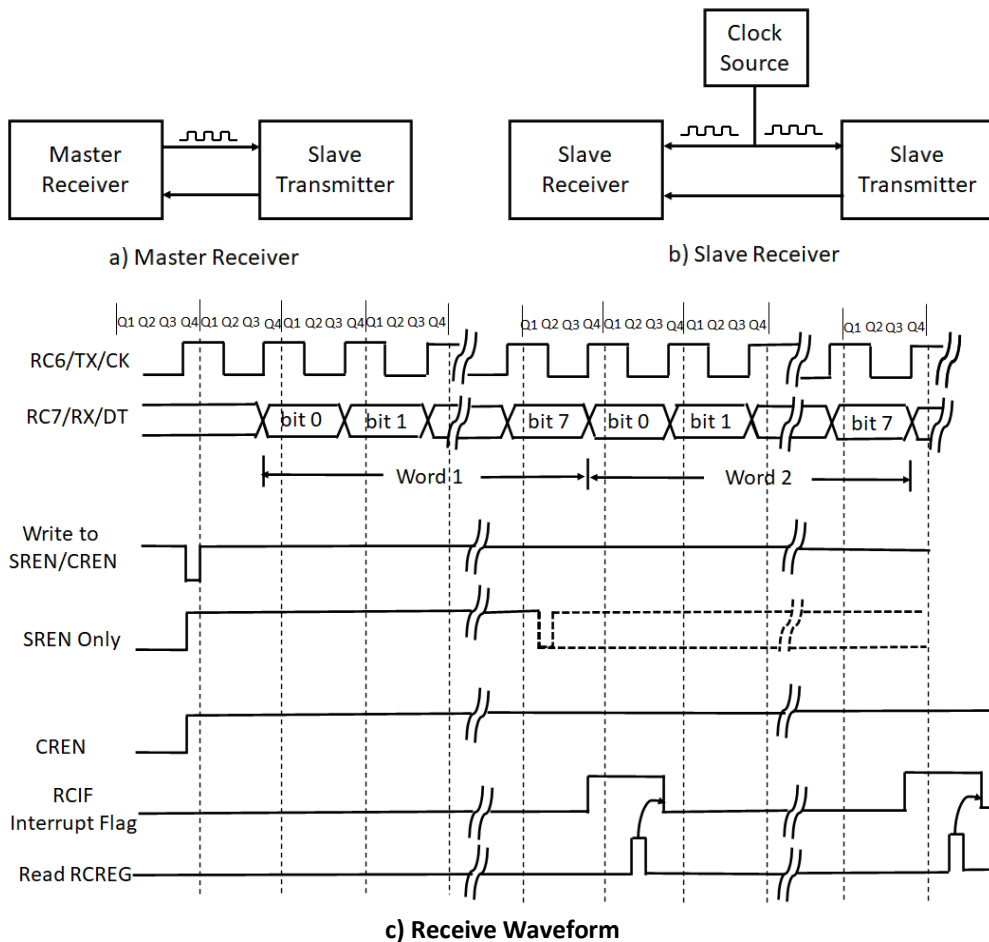


Fig 10.10 USART Synchronous Receive Configurations & Waveform

Note: CREN overrides SREN if both are set

Master Receive:

When the Clock Source Select bit – CSRC: TXSTA<7> is set to 1, the device is configured to be a master. The master generates the synchronous transmission clock on RC6/TX/CK output pin which is controlled through the master's internal baud rate generator

control circuit configured through its SPBRG register. The slave must synchronize itself with the master clock on RC6/TX/CK line and transmit data on the data line – RC7/RX/DT pin. The master samples the RC7/RX/DT pin on the falling edge of the clock to shift the data into RSR register.

Slave Receive:

When the Clock Source Select bit – CSRC: TXSTA<7> is set to 0, the device is configured to be a slave. The RC6/TX/CK pin must now be configured as input so that the external clock is applied at this pin. The synchronous clock generated by the master or an external clock source on RC6/TX/CK pin is used by the slave to synchronize itself and to sample the data of RC7/RX/DT line on the falling edge of the applied clock on RC6/TX/CK line.

The operation of master and slave receive are identical except for master generating the clock and not being able to be put in sleep mode (as it generates the clock), whereas the slave can be put in sleep mode. Additionally, the SREN bit is neglected in slave receive mode. In sleep mode, when the single data byte is available in RCREG register, the RCIF flag would be set and if global enable bits and RCIE is enabled the slave can be woken from sleep mode. The receive waveform is shown in fig 10.10c assuming RC6/TX/CK pin as output while device is a master and input when the device is a slave.

Operation sequence steps for Synchronous Receive – Master/Slave:

Master	Slave
Set the required baud rate by initializing the SPBRG register.	Clock is provided through external source; hence initialization of internal SPBRG register is not required.
The 6th pin of PORTC pin - RC6 must be configured as output for clock transmission by clearing the TRISC<6> bit. The 7 th pin of PORTC pin - RC7 must be configured as input by setting the TRISC<7> bit for data receive.	The 6th pin of PORTC pin - RC6 must be configured as input for clock IN by setting the TRISC<6> bit. The 7 th pin of PORTC pin - RC7 must be configured as input by setting the TRISC<7> bit for data receive.
Enable Synchronous mode by setting SYNC bit - TXSTA<4> to 1; Enable Serial Port by setting SPEN - RCSTA<7> to 1; Set Master mode by setting CSRC - TXSTA<7> to 1.	Enable Synchronous mode by setting SYNC bit - TXSTA<4> to 1; Enable Serial Port by setting SPEN - RCSTA<7> to 1; Set Slave mode by clearing CSRC - TXSTA<7> to 0.
Clear SREN/CREN - RCSTA<5:4> to 0	Clear CREN - RCSTA<4> to 0. SREN is neglected in slave mode.

If interrupts are desired, Enable RCIE bit of PIE1 register to 1 along with global interrupt enables	If interrupts are desired, Enable RCIE bit of PIE1 register to 1 along with global interrupt enables
If 9-Bit receive is desired, set 9-bit receive enable RX9 bit - RCSTA<6> to 1	If 9-Bit receive is desired, set 9-bit receive enable RX9 bit - RCSTA<6> to 1
For multi byte, receive set CREN - RCSTA<4> to 1. Enable single byte or multi byte receive. For single byte receive, set SREN bit - RCSTA<5> to 1; CREN overrides SREN when both are set.	Enable multi byte receive by setting CREN bit - RCSTA<4> to 1. SREN is neglected in slave mode.
The RCIF interrupt flag is set when data receive is completed. Interrupt is generated if RCIE and global interrupts are set	The RCIF interrupt flag is set when data receive is completed. Interrupt is generated if RCIE and global interrupts are set
If 9-bit reception is enabled, read RX9D in RCSTA<0>; determine if error occurred during reception, i.e. OERR or FERR RCSTA<2:1> bits.	If 9-bit reception is enabled, read RX9D in RCSTA<0>; determine if error occurred during reception, i.e. OERR or FERR RCSTA<2:1> bits.
If there is no error, read RCREG for the 8-bit received data.	If there is no error, read RCREG for the 8-bit received data.
If error, clear error by disabling CREN - RCSTA<4> to 0. Then enable CREN by setting CREN - RCSTA<4> to 1	If error, clear error by disabling CREN - RCSTA<4> to 0. Then enable CREN by setting CREN - RCSTA<4> to 1

Program 10.10: Configure PIC18 as a **master** that can synchronously receive serial data till a 0 value is encountered and store data to file location starting from 0x30. Assume crystal oscillator frequency of 11.0592 MHz to transfer at 9600 baud rate with interrupt.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x08	; Set Origin 0x0008 of High Priority interrupt
	BTFSS	PIR1,RCIF	; Check if interrupt is due to Receive interrupt
	RETFIE		; If not due to RCIF return from ISR
	GOTO	RC_ISR	; If interrupt is due to Receive interrupt, go ; to RC_ISR
	ORG	0x50	; Set Origin 0x0050 of RC_ISR
RC_ISR	MOVF	RCREG,0,0	; WREG = RCREG

	TSTFSZ	WREG,0	; Check if the WREG is 0, i.e WREG is ; empty, Skip next instruction if empty
	BRA	RS	; Branch to RS, if WREG is not 0
	BRA	SR	; Branch to SR, if WREG is 0
RS	MOVWF	POSTINC0,0	; [FSR0] = WREG, FSR0 = FSR0 + 1
	NOP		; No Operation
	RETFIE		; Return from RC_ISR
SR	BCF	PIE1,RCIE	; Disable Receive interrupt.
	RETFIE		; Return from RC_ISR
	ORG	0x0100	; Set Origin 0x0100 for Main Program
START			
; For Master RC6 is Output, TRIS<6>=0, Master Generates Clock.			
	BCF	TRISC, CK	; TRISC<6> = 0, Set RC6 as Output for CK ; functionality
	MOVLW	0x11	; WREG = 0x11
	MOVWF	SPBRG	; SPBRG = WREG = 0x11, Set Baud Rate ; 9600 at 11.0592MHz 1 Start, 8 Data, 1 ; Stop Bits.
; For Master or Slave Receive RC7 is Input, TRIS<7>=1, Data Receive			
	BSF	TRISC, DT	; TRISC<7> = 1, Set RC7 as Input for RX ; functionality
	BSF	RCSTA,SPEN	; Enable Serial Port. RCSTA<7> = 1
; For Master Synchronous TXSTA = 0x90			
	MOVLW	0x90	; WREG = 0x90
	MOVWF	TXSTA	; TXSTA = 0x90, Set Master Synchronous Mode
	BSF	RCSTA,CREN	; Enable Continuous Receive, may use SREN ; for single byte transfer which gets disabled ; after 1 transfer
	LFSR	0,0x30	; FSR0 = 0x30
	BSF	PIE1,RCIE	; Enable Receive Interrupt
	BSF	INTCON,PEIE	; Enable Peripheral Interrupt
	BSF	RCON,IPEN	; Enable Priorities
	BSF	IPR1,RCIP	; Set Receive interrupt to high priority.
	BSF	INTCON,GIE	; Enable Global Interrupt
HERE	NOP		; No Operation, Alternate function
	BRA	HERE	; Branch to Here
	END		; End

Program 10.11: Configure PIC18 as a **slave** that can synchronously receive serial data till a 0 value is encountered and store data to file location starting from 0x30. Assume crystal oscillator frequency of 11.0592 MHz to transfer at 9600 baud rate with interrupt.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x08	; Set Origin 0x0008 of High Priority
			; Interrupt
	BTFSS	PIR1,RCIF	; Check if interrupt is due to Receive
			; interrupt flag
	RETFIE		; If not due to RCIF return from ISR
	GOTO	RC_ISR	; If interrupt is due to Receive interrupt, go
			; to RC_ISR
	ORG	0x50	; Set Origin 0x0050 of RC_ISR
RC_ISR	MOVF	RCREG,0,0	; WREG = RCREG
	TSTFSZ	WREG,0	; Check if the WREG is 0, i.e WREG is
			; empty, Skip next instruction if empty
	BRA	RS	; Branch to RS, if WREG is not 0
	BRA	SR	; Branch to SR, if WREG is 0
RS	MOVWF	POSTINC0,0	; [FSR0] = WREG, FSR0 = FSR0 + 1
	NOP		; No Operation
	RETFIE		; Return from RC_ISR
SR	BCF	PIE1,RCIE	; Disable Receive interrupt.
	RETFIE		; Return from RC_ISR
	ORG	0x0100	; Set Origin 0x0100 for Main Program
; For Slave RC6 is Input, TRIS<7>=1, External Source Generates Clock. So no baud rate setting			
START	BSF	TRISC, CK	; TRISC<6> = 1, Set RC6 as Input for CK
			; functionality
	BSF	RCSTA,SPEN	; Enable Serial Port. RCSTA<7> = 1
; For Master or Slave Receive RC7 is Input, TRIS<7>=1, Data Receive			
	BCF	TRISC, DT	; TRISC<7> = 1, Set RC7 as Input for RX
			; functionality
; For Slave Synchronous TXSTA = 0x10			
	MOVLW	0x10	; WREG = 0x10
	MOVWF	TXSTA	; TXSTA = 0x10, Set Slave Synchronous Mode
	BSF	RCSTA,CREN	; Enable Continuous Receive,
			; SREN is neglected in slave mode
	LFSR	0,0x30	; FSR0 = 0x30

	BSF	PIE1,RCIE	; Enable Receive Interrupt
	BSF	INTCON,PEIE	; Enable Peripheral Interrupt
	BSF	RCON,IPEN	; Enable Priorities
	BSF	IPR1,RCIP	; Set Receive interrupt to high priority.
	BSF	INTCON,GIE	; Enable Global Interrupt
HERE	NOP		; No Operation, Alternate function
	BRA	HERE	; Branch to Here
	END		; End